Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Programa de Licenciatura en Ingeniería Electrónica



Implementación de prueba de concepto de arquitectura de lógica reconfigurable sobre FPGA

Informe de Ante Proyecto de Graduación para optar por el título de

Ingeniero en Electrónica con el grado académico de Licenciatura

Luis Diego Sandí Quesada

Asesor: Dr. Pablo Mendoza Ponce

Campus Central

14 de junio de 2026

Declaro que el presente documento de tesis ha sido realizado enteramente por mi persona, utilizando y aplicando literatura referente al tema e introduciendo conocimientos y resultados experimentales propios.

En los casos en que he utilizado bibliografía he procedido a indicar las fuentes mediante las respectivas citas bibliográficas. En consecuencia, asumo la responsabilidad total por el trabajo de tesis realizado y por el contenido del presente documento.

Luis Diego Sandí Quesada

Cartago, 14 de junio de 2026

Céd: 1-1750-0693

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica
Ante Proyecto de Graduación
Acta de Aprobación

Defensa de Proyecto de Graduación
Requisito para optar por el título de Ingeniero en Electrónica
Grado Académico de Licenciatura

El Tribunal Evaluador aprueba la defensa del proyecto de graduación denominado *Implementación de prueba de concepto de arquitectura de lógica reconfigurable sobre FPGA*, realizado por el señor Luis Diego Sandí Quesada y, hace constar que cumple con las normas establecidas por la Escuela de Ingeniería Electrónica del Instituto Tecnológico de Costa Rica.

Miembros del Tribunal Evaluador

M.Sc. Por definir
Profesor Lector

M.Sc. Por definir
Profesor Lector

Dr. -Ing Pablo Mendoza Ponce
Profesor Asesor

Cartago, 14 de junio de 2026

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica
Proyecto de Graduación
Tribunal Evaluador
Acta de Evaluación

Defensa de Proyecto de Graduación
Requisito para optar por el título de Ingeniero en Electrónica
Grado Académico de Licenciatura

Estudiante: **Luis Diego Sandí Quesada** Carné: 2017098994

Nombre del proyecto: *Implementación de prueba de concepto de arquitectura de lógica reconfigurable sobre FPGA*

Los miembros de este Tribunal hacen constar que este proyecto de graduación ha sido aprobado y cumple con las normas establecidas por la Escuela de Ingeniería Electrónica del Instituto Tecnológico de Costa Rica y es merecedor de la siguiente calificación:

Nota de Proyecto de Graduación: _____

Miembros del Tribunal Evaluador

M.Sc. Por definir Profesor Lector	M.Sc. Por definir Profesor Lector

Dr. -Ing Pablo Mendoza Ponce
Profesor Asesor

Cartago, 14 de junio de 2026

Índice general

Índice de figuras III

Índice de tablas IV

1. Introducción 1
- 1.1. Planteamiento del problema . . . 1
  - 1.1.1. Estado Actual . . . 1
  - 1.1.2. Definición del Problema . . . 2
- 1.2. Estructura del Documento . . . 2

2. Marco teórico 3
- 2.1. Lógica Reconfigurable . . . 3
- 2.2. Arquitectura interna de FPGAs . . . 4
  - 2.2.1. Tablas de Consulta . . . 4
  - 2.2.2. Enrutamiento . . . 6
  - 2.2.3. Matriz de conmutación . . . 7
- 2.3. Arquitectura Modulo de Jerarquía Superior . . . 8
  - 2.3.1. Escaneo de Fronteras/JTAG . . . 8
  - 2.3.2. Interfaz del Puerto de Acceso de Prueba . . . 9
  - 2.3.3. Controlador JTAG TAP . . . 10
  - 2.3.4. Registros del JTAG . . . 10
- 2.4. Filtrado digital . . . 11
- 2.5. Estado del Arte . . . 12
  - 2.5.1. eFPGAs . . . 12
  - 2.5.2. Herramientas de software . . . 14

3. Solución Propuesta 15
- 3.1. Primera Etapa . . . 15
- 3.2. Segunda Etapa . . . 15
- 3.3. Tercera Etapa . . . 15
- 3.4. Cuarta Etapa . . . 16

4. Hipótesis y Objetivos del Proyecto 17
- 4.1. Hipótesis . . . 17
- 4.2. Objetivos . . . 17

4.2.1. General 17
4.2.2. Específicos 17
4.3. Lista de Tareas por Objetivo 18
4.4. Alcances y Limitaciones del Proyecto 20

5. Experimento Preliminar **21**
5.1. Metodología del experimento 21
5.2. Resultados Esperados 22

6. Cronograma **23**

Bibliografía **25**

Índice de figuras

2.1. Diagrama simplificado de matriz de programable de compuertas lógicas. [7] 4
2.2. Grafica de LUT de uso optimo. [7] 5
2.3. Diagrama de configuración del enrutamiento y bloques lógicos. [7] 6
2.4. Diagrama de bloques de cables cortos y largos. [7] 7
2.5. Diagrama matriz de conmutación. [7] 8
2.6. Diagrama de la arquitectura TAP. [5] 9
2.7. Diagrama de conexion JTAG y TAP. [6] 9
2.8. Máquina de estados del controlador TAP. [6] 10

6.1. Diagrama de Gantt de las actividades a realizar 24

Índice de tablas

6.1. Lista de tareas del proyecto 23

Capítulo 1

Introducción

La Christian-Albrechts-Universit¨at zu Kiel conocida como CAU y Universidad de Kiel, referente en investigación tecnológica en Alemania, ha iniciado una investigación para diseñar una arquitectura de FPGA propia. A diferencia de las soluciones comerciales de caja negra, este desarrollo busca ofrecer una plataforma competitiva para fines académicos y experimentales. El proyecto ha captado el interés de TSMC, el fabricante de semiconductores más grande del mundo, que ha manifestado su intención de apoyar al centro educativo mediante el acceso a nodos de fabricación avanzados. Esta sinergia se extiende a Centroamérica gracias a los lazos históricos de cooperación entre la Universidad de Kiel y el Instituto Tecnológico de Costa Rica, permitiendo una transferencia de conocimiento de alto nivel en el área de sistemas digitales complejos. La investigación se centra en el concepto de la caracterización de una FPGA embebida en lugar de la fabricación de un IC desde cero. Su desarrollo consta del diseño de una eFPGA en conjunto de un sistema para la inyección de flujo de bits para su correcta programación. La esencia del proyecto es hardware reconfigurable dentro de otro para su caracterización y desarrollo, la implementación de un filtro digital parametrizable se requiere para términos del comportamiento de la eFPGA, entre las métricas a estudiar se encuentra el consumo de potencia, las rutas críticas y la eficiencia de las tablas de consulta.

1.1. Planteamiento del problema

1.1.1. Estado Actual

Aunque existen arquitecturas de FPGA de desarrollo académico, los mismos no cuentan con la capacidad de la flexibilidad de una FPGA comercial debido a características como la cantidad de tablas de consulta, potencia y velocidad de procesamiento, por lo que su desarrollo se ha quedado estancado en su propia academia. Las FPGAs comerciales requieren licencias para su uso además del elevado precio que significa la compra de sufientes para impartir un curso. Además, existe un interés de TSMC para su dessarrollo,

sin embargo, aún no se cuenta con un modelo caracterizado ni validado.

1.1.2. Definición del Problema

La dependencia de arquitecturas de hardware cerrado limita la capacidad de los centros de investigación para innovar en el diseño de semiconductores. En la colaboración CAU TEC, el problema radica en la falta de una metodología validada para caracterizar el rendimiento, área y consumo de una eFPGA. Sin una implementación previa en una FPGA física que actúe como entorno de prueba para aplicaciones como filtros digitales, no es posible asegurar que el diseño final sea competitivo para su futura fabricación con TSMC.

1.2. Estructura del Documento

En el **capítulo 2** se presenta un informe de la investigación del estado del arte actual del problema, así como una descripción de las partes y características del proyecto, Se profundiza en la arquitectura básica de una FPGA y en el método para cargarle el bitstream.

En el **capítulo 3** se presenta las posible solución al problema donde analizamos las etapas a tomar en cuenta para la validación de una arquitectura de lógica reconfigurable, se describe además cada parte del proyecto y los resultados que se esperan.

En el **capítulo 4** se expone la hipótesis planteada, además, se presenta el objetivo general para este trabajo, así como, los objetivos específicos del mismo. Se muestran metas por objetivo propuesto. Por último, se resaltan los alcances y limitaciones que presenta este trabajo.

En el **capítulo 5** se presenta el posible experimento preliminar que detalla la estrategia del proyecto y el enfoque necesario prueba el sistema y el flujo de herramientas a través de un experimento bien diseñado y riguroso.

En el **capítulo 6** se presentan las tareas necesarias a llevar a cabo a lo largo de este trabajo, así como el diagrama de Gantt para poder completar con las tareas propuestas.

Capítulo 2

Marco teórico

2.1. Lógica Reconfigurable

En 1991 nace comercialmente la primera computadora reconfigu rable, la Algotronix CHS2X4. Previo a este hito, la investigación respectiva a la lógica reconfigurable data a la década de 1960 donde Gerald Estrin propuso el concepto pio nero de una computadora con una matriz reconfigurable de hardware, donde su uso se adapta a la necesidad y una vez terminado su objetivo se podría darle propósito con un nuevo objetivo. Posteriormente surgieron investigaciones destacables con arquitectu ras reconfigurables como Copacabana, Matrix, GARP, Elixent, NGEN, Polyp, MereGen, PACT, XPP, Silicon Hive, Montium, Pléyades, Morphosys y PiCoGA, cabe mencionar que la convergencia ante la investigación de las lógica reconfigurable y sus posibles ar quitecturas es una consecuencia del creciente mercado del silicio en los años ochentas y noventas. La idea era efectiva, una pieza de hardware que mediante el software se le pu diese cambiar el funcionamiento, así es como se originan los lenguajes de descripción de hardware. Los lenguajes de descripción de hardware surgen de la idea de simplificar la programación de las arquitecturas reconfigurables, se puede expresar su funcionamiento mediante la descripción del sistema basado en sus entradas, salidas, reiteraciones y fun ciones lógicas/aritméticas. La descripción de hardware da rienda a múltiples niveles de abstracción para generar la misma celda lógica, desde el uso de un operador aritmético, por su construcción mediante compuertas lógicas o su diseño en el nivel de abstracción de celda. Estos niveles de abstracción permiten simplificar el diseño del software de manera que no es necesario describir las conexiones físicas a las compuertas, permitiendo que con la descripción de su funcionamiento el sistema funcione sin la necesidad de explicar sus conexiones internas. Luego de la síntesis del software se espera que entre menor sea el nivel de abstracción utilizado, se minimice al pasarlo al hardware, por lo que la com plejidad a nivel de compuertas será mayor que a nivel funcional. Algunos ejemplos de lenguajes de descripción de hardware son VHDL, Verilog, System Verilog, Slide, Conlan, ISPS, Adlib, Hill, Oode y Boris. La jerarquía del diseño dicta el orden de funcionamiento de los múltiples diseños que pueden ser utilizados por otro diseños, por lo que contar con un diseño básico como una compuerta “and” y que

esta tenga una jerarquía superior al diseño que la va a utilizar generar un problema de jerarquías ya que el diseño que la iba a utilizar no puede encontrarla. Esta característica de los lenguajes de descripción de hardware permite la escalabilidad y modularización de sistemas extensos, en lugar de describir un procesador se pueden describir sus partes y mediante la jerarquía se realiza un diseño superior que una las partes del mismo, de igual manera permite la revisión de todos los componentes para el buen desarrollo del sistema. [3, 7, 8]

2.2. Arquitectura interna de FPGAs

Las FPGAs como su nombre en inglés las describen son Arreglos de compuertas lógicas programables, por lo que su arquitectura se basa en matrices de bloques lógicos, conmuta ciones para el direccionamiento de los datos y el enrutamiento, en la figura 2.1 se observa un diagrama simplificado de la arquitectura interna de una FPGA simple. [3, 7, 8]

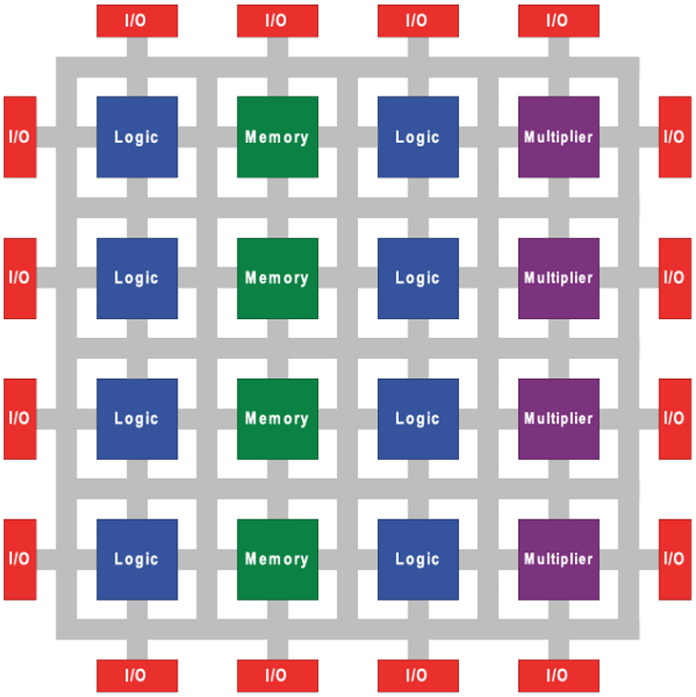


Figura 2.1: Diagrama simplificado de matriz de programable de compuertas lógicas. [7]

2.2.1. Tablas de Consulta

Las tablas de consulta son la herramienta mediante la cual se genera la lógica en una FPGA, en lugar de utilizar compuertas lógicas y tener que redirigir miles de compuertas lógicas, lo que se utiliza son celdas de memoria en conjunto con multiplexores de esta manera se genera una tabla de verdad dentro del sistema mediante un bitstream que lo

configura. También son conocidas como LUTs, según la cantidad de entradas se le añade un número al final a modo de identificación, del número de entradas depende la cantidad de bits de configuración que tiene una LUT en el caso de una LUT4 se tienen 16 posiciones de memoria para cada una de las cuatro entradas y para la salida, en el caso de una LUT6 se tiene 64 bits de configuración. Aunque las LUTs sean escalables es importante analizar las consecuencias de su escalabilidad, ya que existe una curva que describe los cambios en el área y cantidad de LUTs en cierto área. [3, 7, 8]

En la figura 2.2 se muestra que el punto óptimo del tamaño de las LUTs se encuentra cerca de LUT5, no obstante, se aclara que esto no impide tener LUT6 aunque solo se cuente con LUT5, debido a que se va a trabajar con conglomerados de LUTs estos se pueden conectar en paralelo para crear LUTs modulares que representen un LUT6, a forma de ejemplo los LUT5 tienen 32 posiciones de memoria y los LUT6 tienen 64 posiciones de memoria, por lo que al utilizar dos LUT5 se puede generar la lógica necesaria para un LUT6. La práctica anterior es una eficiente solución para evitar el malgasto de área de la FPGA enLUTs. Anivel de implementación física se suele utilizar SRAM para la memoria de las LUTs y estos constituyen los bloques blandos de la FPGA. [7, 1]

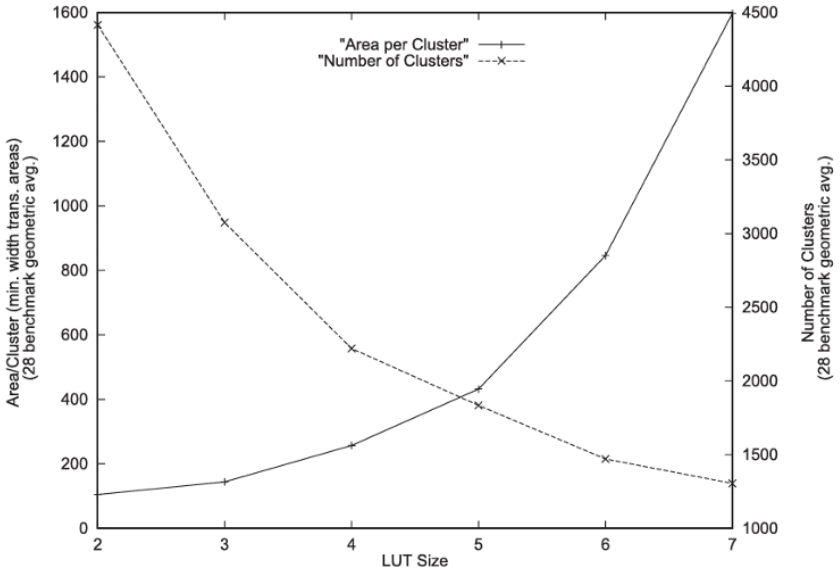


Figura 2.2: Grafica de LUT de uso optimo. [7]

Se debe tomar en cuenta para la eficiencia de una FPGA, no solo tiene lógica reconfigu rable, también cuenta con módulos especializados, en el caso de que toda la lógica sea reconfigurable, se pueden utilizar múltiples LUTs en algo que no es necesario por ejemplo en bloques de multiplicación, este tipo de bloque es conocido como bloque rígido ya que su función es fija. Su implementación es grabada sobre el propio silicio haciéndola no reconfigurable, pero su ventaja se encuentra en su rendimiento ya que al estar sobre el silicio funcionan de manera más rápida que una tabla de consulta y requiere poca área del silicio. [7, 8, 1]

2.2.2. Enrutamiento

La arquitectura del enrutamiento en una FPGA es lo que da conexión a las LUTs al igual que sus entradas, salidas y los bloques rígidos para completar la lógica diseñada. La arquitectura utilizada es la arquitectura tipo isla como se muestra en la figura 2.3. En dicha arquitectura se tienen los bloques lógicos y canales de enrutamiento a los cuatro lados de dichos bloques lógicos, la cantidad n de cables que contiene un canal se define durante la fabricación, sin embargo, es de los parámetros más importantes que puede tener una FPGA ya que define la cantidad de señales que se pueden trabajar de manera simultánea. [3, 7]

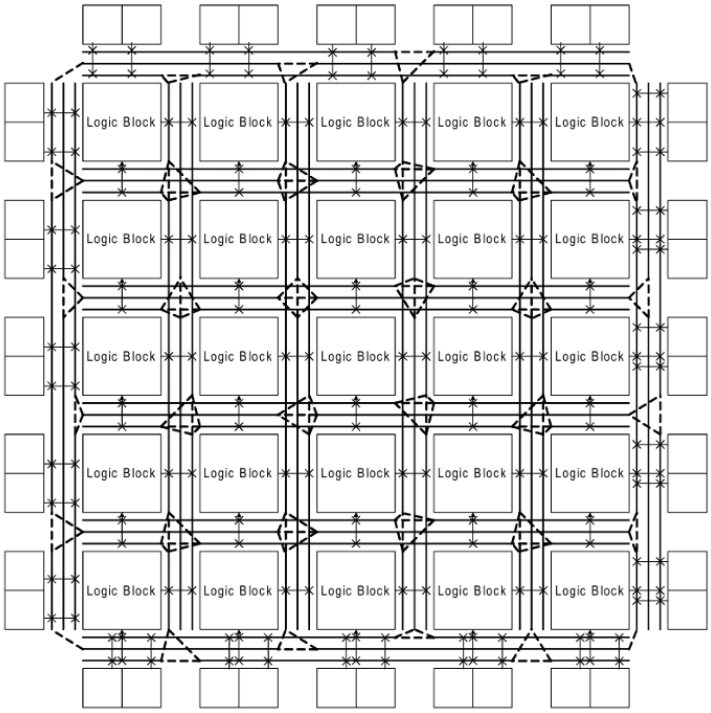


Figura 2.3: Diagrama de configuración del enrutamiento y bloques lógicos. [7]

La lógica de la arquitectura de isla se basa en dos sencillos términos, cables largos y cables cortos, los cables largos componen los canales que se observan en paralelo a los bloques lógicos, su principal función es ser una malla de interconexión entre las señales que se generen o ingresen a la FPGA. Los cables cortos son el medio por el que las señales entran a un bloque desde la malla de enrutamiento o por la que la señal sale de un bloque a la malla de enrutamiento como se muestra en la figura 2.4. [7, 8]

Cabe mencionar que existen arquitecturas de enrutamiento que dependen del orden jerárquico para ordenar la transferencia de datos, pero a manera que la jerarquía aumenta, los retardos aumentan de manera significativa perdiendo el sentido de que existan múltiples niveles jerárquicos y haciendo esta arquitectura contraproducente al objetivo de flexibili dad inherente de una FPGA. La malla de enrutamiento se encarga del desplazamiento de datos en la arquitectura de una FPGA, no obstante, la gestión de los datos es responsa bilidad de la matriz de conmutación. [3, 7]

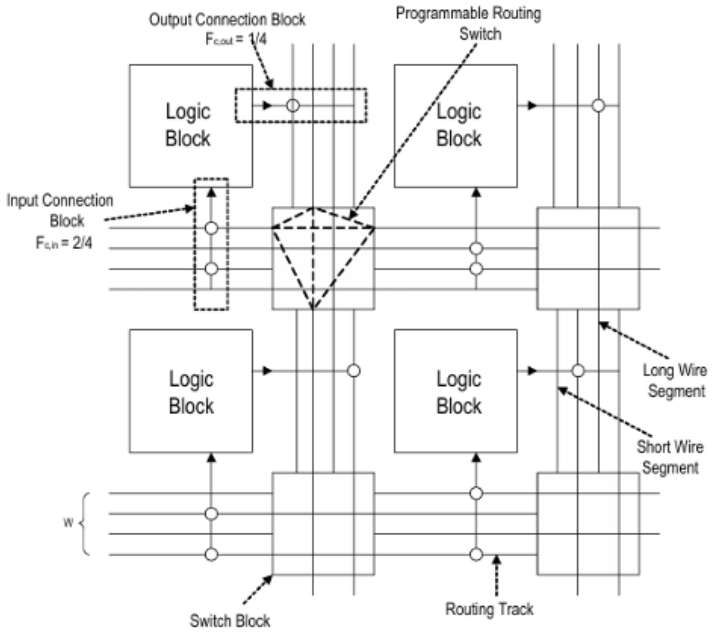


Figura 2.4: Diagrama de bloques de cables cortos y largos. [7]

2.2.3. Matriz de conmutación

La malla de enrutamiento conecta todos los bloques de la arquitectura, pero aún se deben dirigir dichos datos, para que sean la entrada de un bloque o inclusive para que dos bloques trabajen los datos en paralelo, la matriz de conmutación es el mecanismo mediante el cual se ordenan las pistas de cables y se define a las señales cuál bloque es su próximo ingreso. Estas rutas se definen desde que se carga el bitstream a la FPGA, por lo que las rutas son flexibles previas al bitstream, pero estáticas una vez programadas. Su funcionamiento se caracteriza por el uso de multiplexores para definir la ruta, sin embargo también se implementa la lógica necesaria mediante transistores de paso para definir su ruta. [3, 7]

Se considera la intersección de dos canales como una matriz de conmutación, en dichos canales donde cada cable interseca con otro cable se debe considerar la posibilidad de que la señal viaje sobre el mismo cable en la misma dirección inicial, que siga una ruta ascen dente por el cable con el que interseca, o una ruta descendente por el cable que interseca. Para definir la ruta específica que se desea, se requieren de al menos seis transistores de paso para aislar la ruta en el caso de que no se desee bifurcar la señal, las ubicaciones específicas para dichos transistores de paso son conectando la sección de cable izquierdo con el derecho, conectando la sección ascendente con la descendente, la izquierda con la ascendente, la izquierda con la descendente, la ascendente con la derecha y la descendente con la derecha, de esta manera se tiene un control total sobre la dirección deseada de la señal, su control se basa en los datos guardados en una memoria SRAM que requiere seis bits de control, considerando un bit por transistor de paso y una SRAM por intersección de los cables largos tal y como se muestra en la figura número. [3, 7]

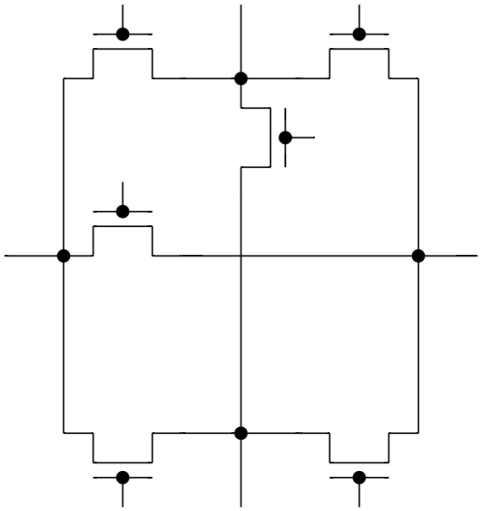


Figura 2.5: Diagrama matriz de conmutación. [7]

2.3. Arquitectura Modulo de Jerarquía Superior

Considerando el funcionamiento de una FPGA, es necesario tener un conocimiento del proceso entre el diseño y la descripción de hardware, para cualquier sistema, esto debido al flujo de bits necesario para dar propósito a la FPGA. Se debe implementar un inyector del flujo de bits además de la FPGA embebida en el nivel superior de jerarquía, de otra manera la FPGA embebida se mantiene en un estado sin programar. Se requiere algún tipo de memoria que se pueda utilizar para luego serializar y cargarla a la FPGA embebida. Las ROM cumplen con características específicas para probar el funcionamiento de la FP GAembebida en conjunto con lógica secuencial que permite la serialización de los datos de la ROM para su posterior inyección en el sistema, de esta implementación suele encargarse el software de diseño en conjunto con los puertos del hardware para la configuración. [3, 7, 8]

2.3.1. Escaneo de Fronteras/JTAG

El Joint Test Action Group, fue desarrollado para resolver problemas en el área de testing de circuitos integrados, esto debido a que los circuitos integrados cada vez disminuían el tamaño y los pines de prueba que se utilizaban acaparaban mucha área del circuito integrado por lo que se investigaban nuevos métodos de probar la funcionalidad de los circuitos integrados. El estándar IEEE 1149.1 surge en 1990 y define la arquitectura TAP en conjunto con el JTAG para diagnóstico, manufactura, validación, caracterización y debug de los circuitos integrados. [9, 2, 16, 5, 6, 4]

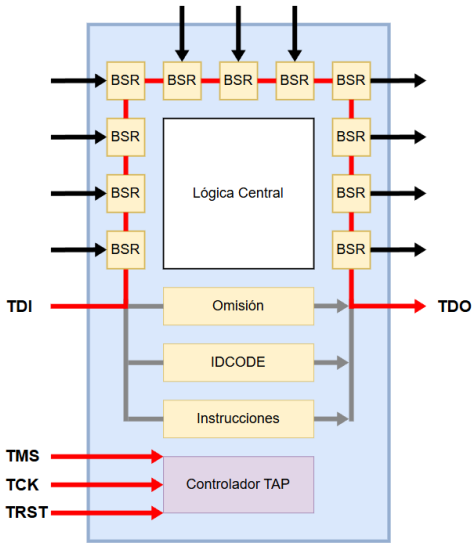


Figura 2.6: Diagrama de la arquitectura TAP. [5]

2.3.2. Interfaz del Puerto de Acceso de Prueba

Su conexión física se basa en 4 pines, TDI (Entrada de datos serial), TMS (Control), TCK (Reloj), TDO (Salida de datos serial) y en ocasiones se añade un pin extra TRST (Reinicio asíncrónico), estas cuatro o cinco señales son conocidas como TAP (Puertos de acceso para pruebas). En el contexto de las FPGAs , se utiliza el JTAG para acceder a la configuración de la memoria para programar el dispositivo, para visualizar la lógica programable del núcleo y el ruteo de los recursos. Esto se ejemplifica en las secciones pasadas dedicadas a explicar la arquitectura de lógica reconfigurable de las FPGAs, donde los registros SRAM configuran las tablas de búsqueda, los caminos que deben seguir los recursos y la disposición de la matrices de conmutación. [9, 2, 5, 6] [4]

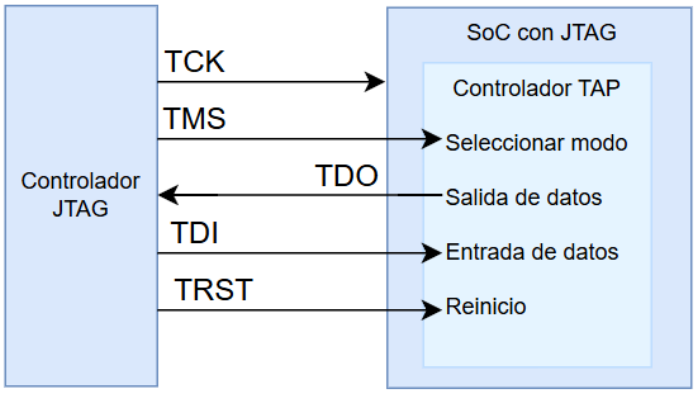


Figura 2.7: Diagrama de conexion JTAG y TAP. [6]

2.3.3. Controlador JTAG TAP

El controlador TAP es el mecanismo de control base del estándar IEEE 1149.1, en este se define como los datos son ciclados a través del dispositivo y determina si se accede a alguno de los registris de datos e/o instruciones. Las señales del controlador se acceden mediante la interfaz TAP explicada anteriormente. El pin de mayor significado para el codntrolador TAP es el TMS debido a que es la entrada (un solo bit) de la máquina de estados que define el camino a tomar de los 16 estados existentes. [9, 2, 5, 6, 4]

Al ordenar los estados en funcionalidad, se obtinen 3 subcategorías:

- Prueba de lógica de reinicio y Marcha/Inactivo.
- Estados de accedo al registro de insrucciones (IR).
- Estados de acceso a los registros de datos (DR).

[9, 2, 5, 6, 4]

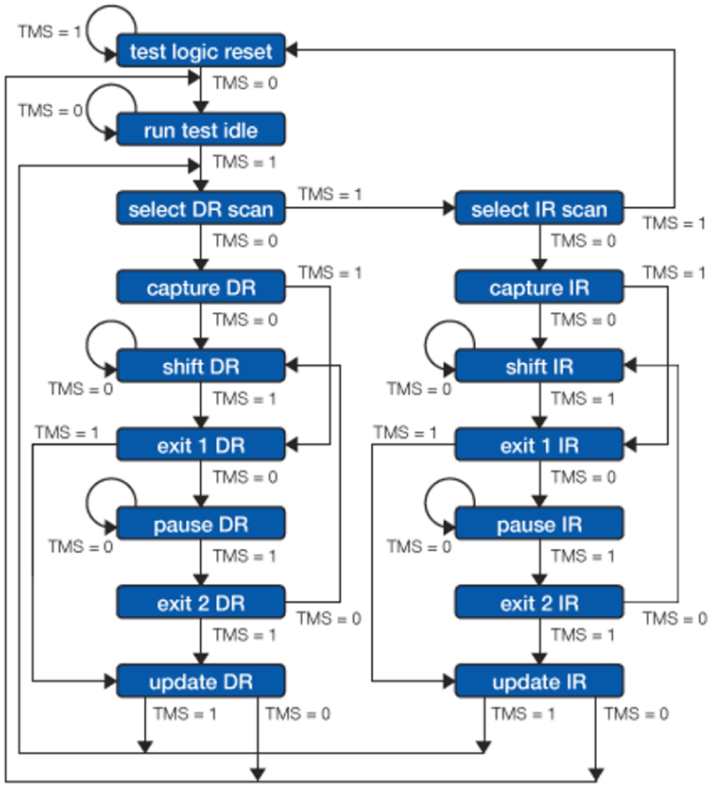


Figura 2.8: Máquina de estados del controlador TAP. [6]

2.3.4. Registros del JTAG

El estándar IEEE 1149.1 define dos tipos de registros, los registros de datos y los registros de instrucciones. [9, 2, 5, 6, 4]

Registros de datos

Los registros de datos incluyen:

- El registro de escaneo de frontera (BSR).
- El registro de omisión (BYPASS).
- El registro del código de identificación (IDCODE).

El controlador TAP se encarga de definir a cuál se accesa mediante la máquina de estados ya que solo se puede utilizar un registro de datos a la vez. [9, 2, 5, 6, 4]

El BSR no es un registro de un solo bloque, sino el una cadena de bloques que funcioan como registro de corrimiento para forzar valores a pines específicos internamente en el SoC, de igual manera se puede utilizar como un muestreo de los valores de los pines considerando que son solo de lectura para realizar un censado. [9, 2, 5, 6, 4]

El registro de omisión solo cuenta con un bit de logitud, y su función es habilitar o desconectar el uso del registro BSR, funciona como paso a la entrada de otros SoC que se quieren probar en serie con el primero, esto permite la optimización del flujo de bits. [9, 2, 5, 6, 4]

El registro del código de identificación es un registro de 32 bits que contiene la identificación única del dispositivo, está grabada en el silicio para evitar su modificación, sirve para que el software sea capaz de identificar el hardware y utilice valores correctos de tensión para el registro BSR y la correcta cantidad de bits. [9, 2, 5, 6, 4]

2.4. Filtrado digital

Basado en la funcionalidad de la transformada discreta de Fourier como representación finita en el dominio de la frecuencia. El filtro digital es un filtro matemático que realiza una convolución, que matemáticamente se puede explicar secuencialmente como un des plazamiento temporal, la multiplicación de la señal por otra señal de referencia y la suma de los resultados para generar el muestreo de la salida. Los filtro digitales se dividen en dos tipos:

- Filtros digitales de respuesta finita al impulso: Estos filtros tienen una respuesta finita al impulso, lo que significa que su salida se vuelve cero después de un número finito de muestras. Son estables y tienen una fase lineal, lo que los hace ideales para aplicaciones donde la distorsión de fase es un problema.[10]
- Filtros digitales de respuesta infinita al impulso: Estos filtros tienen una respuesta infinita al impulso, lo que significa que su salida no se vuelve cero después de un

número finito de muestras. Son más eficientes en términos de recursos computacio nales, pero pueden ser inestables y tener una fase no lineal.[10]

Un ejemplo de un tipo de filtro digital es el filtro de coseno alzado, utiliza como referencia un coseno y su principal fortaleza es reducir la interferencia entre símbolosonfiabilidad de la transmisión de datos. El filtro de coseno alzado se utiliza en sistemas de comunicación digital donde los símbolos pueden superponerse debido a la dispersión del canal. [10]

2.5. Estado del Arte

2.5.1. eFPGAs

El concepto de una FPGA embebida no es nuevo, las FPGAs se utilizan como solución an tes piezas de hardware caras cuyo diseño no es complejo a nivel de características como la potencia requerida o almacenamiento significativo. Su uso se encuentra incluso vinculado a ASICs ya que también significa ahorro para los productores de dicho ASIC, no obstante, en casos de eFPGAs en ASICs, las eFPGAs cuentan con características específicas para la pieza de hardware que se desea implementar en conjunto con el ASIC, por lo que no se espera una gran cantidad de tablas de consulta, menores velocidades de reloj para lógica secuencial, e incluso puede que no se cuente con bloques rígidos especializados a menos que el ASIC lo requiera. Algunos ejemplos de empresas a continuación.[11]

Flex Logic es una compañía tecnológica que utiliza la tecnología de eFPGA desde el 2015, los motiva la reutilización de un circuito integrado luego de su fabricación. Cuentan con un núcleo de eFPGA conocido como FLX10 con 5-7 capas de metal, sus características principales son:

- 20K bits de RAM
- 122500 LUTs 6
- Coarse Grain

Mental trabaja con eFPGA desde el 2009, al igual que Flex Logic su objetivo es la modificación de hardware post producción. La eFPGA de Menta cuenta con sus propias herramientas para la compilación, diseño en RTL y configuración del eFPGA llamada Origami.[11]

- 200000 LUTs 4
- Fine Grain

Achronix trabaja con eFPGA desde el 2016 su modelo de Speedcore eFPGA inlcuye bloques de RAM, y bloques de procesamiento de señales digitales. Cuenta con el software ACE para el diseño de herramientas, su principal uso es como acelerador de hardware y coprocesador reconfigurable.[11]

- 1000 LUTs
- Coarse Grain

NanoXplore es una compañía que data del 2014 cuenta con la eFPGA NX-eFPGA que se puede configurar mediante la aplicación NanoXmap para sintetizar RTL hasta el flujo de bits, el uso de tablas de consulta de 4 entradas le permite utilizar menor área para la eFPGA en la ASIC predeterminada.[11]

- 750000 LUTs
- Coarse Grain

La academia también presenta varias arquitecturas de eFPGA entre ellas:

- FASE eFPGA
- EFPGA Based on Multistage Switching Networks
- ACLBs eFPGA
- Overlay eFPGA
- EFPGAs based on a flexible architecture
- ESL Re-programmable eFPGA
- EFPGA Subsystem
- LIP6 eFPGA
- FlexTiles

No se detallan sus características debido a que la mayoría utiliza LUTs de entre 4 y 6 entradas. Su uso es similar, sustituir una ASIC costosa en la implementación de un proyecto o para investigación de su desarrollo.[11]

2.5.2. Herramientas de software

FABulous es un generador de FPGA de código libre que implementa herramientas como Yosys y nextpnr para el flujo de diseño, cuenta con la licencia Apache 2.0 para uso comercial y académico. Cuenta con cierre del diseño de TSMC de 180nm, Skywater dde 130nm, IHP SG13G2, GF180MCU y 28nm CMOS.[14]

Yosys es una herramienta de código abierto para síntesis de Verilog RTL, tiene la licencia ISC.[15]

- Procesa casi cualquier diseño sintetizable en Verilog-2005.
- Mapeo a librerías de celdas estándar para ASICs.
- Mapeo para FPGAs Xilinx Serie 7 y Lattice iCE40 y ECP5.

Openlane es un traductor de RTL a GDSII basado en múltiples componentes como Yosys, Magic, Netgen, CVC, SPEF-Extractor, Kayout entre otros, y es de código abierto. [13]

Capítulo 3

Solución Propuesta

Este proyecto propone el desarrollo de una metodología de emulación física y caracterización para superar las limitaciones del hardware cerrado y disminuir los riesgos de la fabricación en silicio sin el previo modelado del sistema necesario. Para esto, se proponen cuatro etapas para la solución.

3.1. Primera Etapa

La primera etapa de la solución propuesta para el proyecto se enfoca en la selección de una arquitectura de lógica reconfigurable de código abierto parametrizable, y su implementación mediante un banco de pruebas para comprobar su funcionamiento correcto. De esta manera se elimina la dependencia del flujo de herramientas de un fabricante específico y se asegura la portabilidad del proyecto a diferentes plataformas, al igual que se garantiza la continuidad del proyecto en el caso de querer profundizar en el futuro.

3.2. Segunda Etapa

La segunda parte de la solución propuesta para el proyecto se centra en la instanciación de la arquitectura de lógica reconfigurable seleccionada en la primera etapa sobre una FPGA comercial. Mediante esta etapa se busca un medio para extraer en próximas etapas las métricas de rendimiento y uso de recursos como el área equivalente, el consumo de potencia y los retardos de las rutas críticas.

3.3. Tercera Etapa

La tercera fase de la solución propuesta para el proyecto busca implementar dos circuitos digitales que estresen la arquitectura de lógica reconfigurable seleccionada en la primera

etapa de maneras diferentes. Para esto se propone la implementación de un filtro digital de orden bajo y un multiplicador de enteros. De esta manera se busca obtener métricas de rendimiento para su comparación y se garantiza la funcionalidad del flujo de síntesis y ruteo de la arquitectura de lógica reconfigurable seleccionada.

3.4. Cuarta Etapa

La cuarta etapa de la solución propuesta para el proyecto tiene como objetivo la consolidación de los datos obtenidos en la tercera etapa observando cómo las métricas de rendimiento se compensan entre sí, para determinar si la arquitectura de lógica reconfigurable seleccionada es adecuada para su uso en aplicaciones de alto rendimiento o si es necesario realizar modificaciones según la aplicación en la que se utilice. Esta etapa se enfoca en la interpretación de los datos y cómo su análisis puede guiar el diseño de futuras arquitecturas de lógica reconfigurable para su impresión en silicio.

Capítulo 4

Hipótesis y Objetivos del Proyecto

En este capítulo se presenta la base de la investigación: la hipótesis y las interrogantes derivadas de la misma. El objetivo general consiste en validar esta premisa, para lo cual se han definido objetivos específicos que abordan diferentes aspectos del tema. Asimismo, se describen las tareas precisas requeridas para alcanzar cada objetivo y lograr resultados concluyentes. Por último, se exponen las limitaciones del estudio.

4.1. Hipótesis

"La variación sistemática de los parámetros estructurales en una arquitectura reconfigurable de hardware abierto permite identificar una configuración óptima que maximiza el rendimiento de temporización para un conjunto específico de circuitos benchmark, demostrando que la arquitectura de la red de interconexión es el factor dominante en el consumo de potencia y área sobre los bloques lógicos."

4.2. Objetivos

4.2.1. General

Evaluar el impacto de la variación de parámetros estructurales en el rendimiento, área y potencia de una arquitectura abierta de lógica reconfigurable a nivel de simulación y pruebas físicas utilizando una FPGA comercial como plataforma de emulación física.

4.2.2. Específicos

- Seleccionar una arquitectura de lógica reconfigurable de hardware abierto y su flujo de herramientas asociado, justificando la elección basado en su nivel de parametri-

zación, soporte de verificación funcional e historial de implementación física en otros proyectos.

- Configurar e instanciar la arquitectura seleccionada en una tarjeta FPGA y caracterizarla basandose en reportes de síntesis, ruteo y ejecución funcional, además de implementar al menos dos circuitos de prueba en dicha lógica programable.
- Caracterizar el impacto de la variación de los parámetros estructurales disponibles en la arquitectura de lógica programable sobre la utilización de recursos, el consumo de potencia y el rendimiento a nivel de temporización.

4.3. Lista de Tareas por Objetivo

Esta sección se enfoca en proporcionar un desglose detallado de las tareas que deben realizarse para lograr cada objetivo específico en el proyecto. Esta estructura por objetivo facilita la comprensión de las tareas individuales en el contexto de los objetivos más amplios del proyecto. Cada objetivo del proyecto tendrá su propia lista de tareas relacionadas, lo que proporciona una visión clara de lo que se debe lograr para alcanzar ese objetivo. Estas tareas se presentan de manera secuencial, lo que indica el orden en el que deben completarse.

Objetivo Específico 1

Este objetivo se centra en la investigación de la arquitectura de lógica reconfigurable y el flujo de harramientas para su implementación, considerando la el soporte para su parametrización, verificación funcional e historial de implementación física.

1. Investigar y recopilar información sobre las distintas arquitecturas de lógica reconfigurable de hardware abierto disponibles.
2. Evaluar la flexibilidad y el nivel de parametrización estructural que ofrece cada arquitectura preseleccionada.
3. Analizar la madurez del flujo de herramientas asociado, comprobando la compatibilidad y el soporte para la verificación funcional.
4. Revisar el historial de implementación física en silicio de las opciones evaluadas para constatar su viabilidad y confiabilidad en proyectos previos.
5. Seleccionar la arquitectura definitiva y justificar técnicamente la elección, basada en el cumplimiento de los criterios previamente analizados.

Las tareas descritas justifican el uso de la arquitectura en las próximas etapas y su implementación en la FPGA.

Objetivo Específico 2

El segundo objetivo se centra en la configuración de la arquitectura seleccionada mediante JTAG y la caracterización basada en reportes desplegados por las herramientas seleccionadas en la etapa anterior, además de la implementación de dos circuitos de prueba.

1. Ajustar los módulos de la arquitectura seleccionada para asegurar su compatibilidad con la herramienta de síntesis de la FPGA objetivo.
2. Integrar la arquitectura en el proyecto principal de la herramienta de desarrollo.
3. Correr el proceso de síntesis y extraer los reportes de la cantidad de LUTs, flip-flops y bloques de memoria requeridos.
4. Ejecutar la implementación en la FPGA y compilar los reportes de tiempos de setup y hold, consumo de potencia estimado y congestión de ruteo.
5. Generar el bitstream necesario para configurar la FPGA física.
6. Seleccionar dos circuitos que estresen diferentes aspectos de la lógica programable.
7. Codificar ambos circuitos independientemente.
8. Escribir los testbenches correspondientes y simular el comportamiento de los circuitos de prueba para asegurar su corrección funcional antes de la implementación física.
9. Compilar y cargar el primer y segundo circuito de prueba sobre la arquitectura de lógica programable ya instanciada.
10. Estimular las entradas en el hardware real, capturar las salidas y contrastar la ejecución funcional contra los resultados obtenidos en las simulaciones

Estas tareas permiten validar la arquitectura seleccionada y dan información benéfica para el desarrollo de la caracterización de la arquitectura seleccionada.

Objetivo Específico 3

El tercer objetivo da base a la caracterizacion de la arquitecura de lógica programable seleccionada según la utilización de recursos.

1. Identificar cuáles variables de la arquitectura de la arquitectura lógica programable serán modificables.
2. Definir un conjunto discreto de valores para cada parámetro seleccionado y limitar las combinaciones posibles a un número manejable y representativo.
3. Recopilar la cantidad de bloques lógicos básicos utilizados y la estimación de área equivalente de silicio.

4. Extraer del Análisis de Temporización Estática y el retardo de la ruta crítica.
5. Compilar las estimaciones de consumo de potencia dinámica y estática reportadas tras el ruteo.
6. Procesar los datos consolidados para graficar el comportamiento de las métricas frente a la variación de cada parámetro.
7. Analizar los puntos donde la mejora de una métrica perjudica a otra (Trade-off).
8. Concluir cuáles son las configuraciones de parámetros estructurales que ofrecen el mejor balance técnico para los casos de uso proyectados.

4.4. Alcances y Limitaciones del Proyecto

En la sección de limitaciones se exponen los desafíos y restricciones propios de la naturaleza del proyecto. Si bien estas condiciones pueden representar obstáculos, al mismo tiempo delimitan el alcance del trabajo y establecen un marco de referencia realista para su ejecución. A continuación, se detallan las principales limitaciones identificadas:

- La ejecución de los experimentos se llevará a cabo con la FPGA disponibles durante la evaluación del proyecto. Esta limitación en los recursos físicos para la realización de pruebas podría restringir tanto la diversidad de los escenarios evaluados como la naturaleza de los datos susceptibles de recolección.
- Al instanciar una arquitectura de lógica programable sobre una FPGA comercial, el área, la potencia y el retardo de temporización estarán limitados por la arquitectura de la FPGA anfitriona.
- La configuración óptima identificada estará restringida a las características de los circuitos seleccionados y podría no ser generalizable a todas las aplicaciones digitales.
- No se realizará una exploración exhaustiva de todo el espacio de diseño, por lo que la configuración óptima que maximiza el rendimiento corresponderá únicamente a un óptimo dentro del subconjunto explorado.
- Las métricas de consumo de potencia se extraerán a partir de reportes generados por herramientas de síntesis y ruteo, no mediante mediciones físicas directas sobre la arquitectura implementada por lo tanto se introduce un margen de error inherente a los modelos matemáticos y estimaciones.

Capítulo 5

Experimento Preliminar

En esta sección se especifica la estrategia planificada para realizar un experimento preliminar que permita validar la metodología propuesta para la emulación física y extracción de métricas de rendimiento en arquitecturas de lógica reconfigurable. Se escaló el experimento a un tamaño pequeño para asegurar su viabilidad en el tiempo del proyecto y su funcionamiento correcto, lo que permitirá realizar ajustes a la metodología antes de su aplicación a un sistema más complejo. El principal interés del experimento es garantizar la correcta síntesis, mapeo, reteo y generación del bitstream de la arquitectura lógica reconfigurable mediante un circuito digital de prueba básico sin intervención de hardware físico.

5.1. Metodología del experimento

Se diseñará mediante código RTL una microarquitectura de lógica programable por un solo grupo de cuatro tablas de búsqueda, matrices de conmutación y su ruteo respectivo, todo con sus registros para asegurar que sea reconfigurable.

Como circuito de referencia para emular dentro de la microarquitectura reconfigurable descrita, se utilizará un multiplexor de dos entradas, se describirá de manera estructural compuerta por compuerta como se muestra en la función lógica 5.1. Dicho multiplexor es compuesto por dos compuertas AND, una compuerta NOT y una compuerta OR, cada una de las compuertas será asignada a una de las LUTs disponibles en la microarquitectura reconfigurable.

$$Y = (A \cdot \overline{S}) + (B \cdot S) \tag{5.1}$$

Fidnalmente se realizará una simulación funcional y de temporización utilizando un banco de pruebas en software para verificar que las señales de entrada conmuten correctamente hacia la salida según el estado de la línea de selección.

5.2. Resultados Esperados

Por este medio se pretende:

- Confirmar que las herramientas abiertas son capaces de reconocer la estructura de la microarquitectura reconfigurable y mapear la lógica combinacional del multiplexor.
- Verificar que el software puede interconectar las LUTs de manera exitosa para propagar la señal del multiplexor.
- Obtener los primeros reportes de retardo estimados por el software, sirviendo como línea base para la metodología de caracterización final.

Capítulo 6

Cronograma

En la Tabla 6.1 se muestra las actividades a desarrollar durante la investigación y elaboración del proyecto. La Figura 6.1 muestra el diagrama de Gantt correspondiente a las actividades.

Tarea #	**Tarea a realizar**
1	Investigación sobre las arquitecturas de lógica reconfigurable
2	Configuración del entorno de desarrollo y herramientas
3	Revisión y ajuste de los módulos en código RTL
4	Ejecución de los primeros procesos de síntesis lógica
5	Generación del archivo de configuración
6	Descripción de hardware de los dos circuitos de prueba
7	Procesamiento del filtro digital y el multiplicador de enteros
8	Extracción de las métricas finales
9	Análisis crítico de los puntos de trade-off
10	Elaboración del informe escrito
11	Entrega del informe final
12	Defensa del proyecto

Tabla 6.1: Lista de tareas del proyecto

En la siguiente figura 6.1, presentamos un Diagrama de Gantt que muestra gráficamente el cronograma de nuestro proyecto. Este diagrama nos permite visualizar con claridad la planificación de las diferentes actividades y su duración a lo largo de las semanas. Las tareas de cada objetivo específico están representadas por barras horizontales, cuya longitud es proporcional a la duración estimada de cada tarea. La organización de estas barras a lo largo de la línea temporal proporciona una perspectiva clara de cuándo y cómo se llevarán a cabo las actividades, facilitando así la gestión del tiempo y la distribución de los recursos. Las superposiciones indican la simultaneidad de tareas y las intersecciones demuestran los puntos donde las actividades están interconectadas. Este diagrama es la hoja de ruta visual de nuestro proyecto, y nos ayudará a mantenernos en la senda correcta, asegurando que todas las tareas se completen en los plazos establecidos.

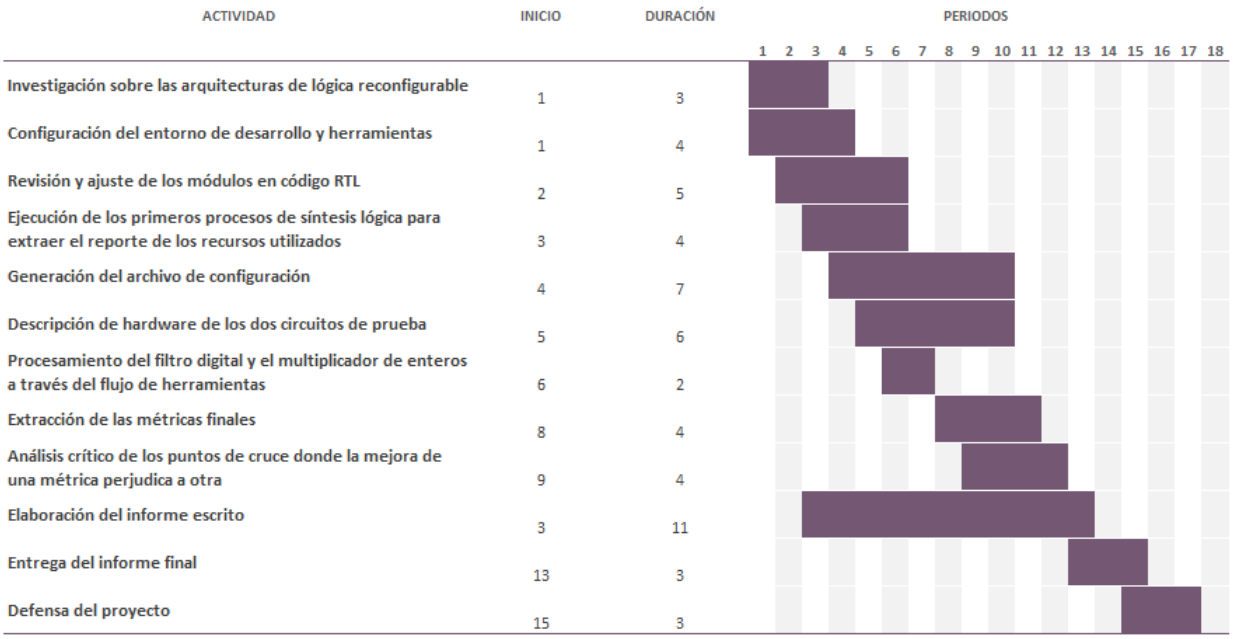


Figura 6.1: Diagrama de Gantt de las actividades a realizar

Bibliografía

[1] Nassim Abderrahmane. «Maximize Flexibility in SoC Design with eFPGAs». En: *Electronic Design* (). URL: https://www.electronicdesign.com.

[2] José Roberto Calvo Monge. «Testing y Pruebas Funcionales. Sesión 4: Interfaces básicas de testing/funcionales en un SOC». Diapositivas del programa Técnico en Semiconductores. Material de clase. 2026.

[3] Fatemeh Serajeh Hassani et al. «An Efficient FPGA Architecture with Turn-Restricted Switch Boxes». En: *ACM Trans. Des. Autom. Electron. Syst.* (2024). ISSN: 1084-4309. DOI: 10.1145/3643809. URL: https://doi.org/10.1145/3643809.

[4] *IEEE Standard for Test Access Port and Boundary-Scan Architecture*. IEEE Computer Society, 2013. DOI: 10.1109/IEEESTD.2013.6515989.

[5] JTAG Technologies. *Boundary Scan*. JTAG Technologies. URL: https://www.jtag.com/boundary-scan/.

[6] JTAG Technologies. *How the JTAG TAP Controller Works*. JTAG Technologies. URL: https://www.jtag.com/how-the-jtag-tap-controller-works/.

[7] Ian Kuon, Russell Tessier y Jonathan Rose. «FPGA Architecture: Survey and Challenges». En: *Foundations and Trends in Electronic Design Automation* (2007). DOI: 10.1561/1000000005. URL: https://doi.org/10.1561/1000000005.

[8] Uwe Meyer-Baese. *Digital Signal Processing with Field Programmable Gate Arrays*. 4.ª ed. Springer, 2014. DOI: 10.1007/978-3-642-45309-0. URL: https://doi.org/10.1007/978-3-642-45309-0.

[9] Victor P. Nelson. *Testing and Boundary Scan*. ELEC 4200 Lecture Slides. Auburn University. URL: http://www.eng.auburn.edu/~nelson/courses/elec4200/Slides/BoundaryScan.pdf.

[10] John G. Proakis y Dimitris G. Manolakis. *Digital Signal Processing: Principles, Algorithms, and Applications*. 3.ª ed. Prentice Hall International, 1996. ISBN: 0-13-394338-9.

[11] Hajer Saidi et al. «A Survey and Comparative Study of Embedded FPGA Architectures». En: *IEICE Transactions on Information and Systems* E101.D (2018). DOI: 10.1587/transinf.2017PAP0010. URL: https://doi.org/10.1587/transinf.2017PAP0010.

[12] Yizhou Shan. *FPGA Bitstream Explained*. Oct. de 2019. URL: http://lastweek.io/fpga/bitstream/.

[13] The OpenROAD Project. *OpenLane: An Automated RTL to GDSII Flow*. GitHub repository. 2026. URL: https://github.com/The-OpenROAD-Project/OpenLane.

[14] TheFABulousProject Authors. *FABulous: Framework for Architecture Synthesis and Layout of Open User-customizable Scaffolding*. Ver. latest. University of Manchester, 2024. URL: https://fabulous.readthedocs.io/en/latest/.

[15] Claire Wolf. *Yosys Open Synthesis Suite*. 2024. URL: https://yosyshq.net/yosys/.

[16] XJTAG. *eBook: ¿Qué es JTAG?* Guía técnica en PDF. XJTAG. URL: https://www.xjtag.com/wp-content/uploads/xjtag-ebook_what-is-jtag-es.pdf.